

UNIVERSIDAD MILITAR NUEVA GRANADA



Laboratorio 5. Programación con hilos en Python

Guías de Prácticas de Laboratorio	Identificación: GL-AA-F-1	
	Número de Páginas: 9	Revisión No.: 2
	Fecha Emisión: 2018/01/31	
Laboratorio de: Programación 3		
Titulo de la Práctica de Laboratorio: Programación con Hilos en Python		

Elaborado por: Ing. Ricardo Castillo, Ph.D. Docente tiempo completo Ing. Nathaly Sánchez, M.Sc. Docente hora cátedra	Revisado por: Ing. Luís Francisco Niño, M.Sc Jefe de área robótica	Aprobado por: Ing. Dario Amaya, Ph.D. Director de Programa
---	---	---

UNIVERSIDAD MILITAR NUEVA GRANADA



Laboratorio 5. Programación con hilos en Python

Control de Cambios

Descripción del Cambio	Justificación del Cambio	Fecha de Elaboración / Actualización
Actualización del formato		2021/04/19
Actualización del procedimiento	La temática no cambia, se ajusta el entregable para evitar plagios	2024/07/12
Actualización del procedimiento	La temática no cambia, se ajusta el entregable para evitar plagios	2025/17/01



Laboratorio 5. Programación con hilos en Python

1. **FACULTAD O UNIDAD ACADÉMICA:** Ingeniería

2. **PROGRAMA:** Ingeniería Mecatrónica

3. **ASIGNATURA:** Programación 3

4. **SEMESTRE:** III

5. **OBJETIVO:**

General: Desarrollar aplicaciones básicas que usen el concepto de programación por hilos

Específicos:

- Conocer las características básicas de la programación por hilos
- Independizar procesos en un programa mediante hilos
- Identificar las ventajas de la programación por hilos

6. **MATERIALES, REACTIVOS, INSTRUMENTOS, SOFTWARE, HARDWARE O EQUIPOS DEL LABORATORIO:**

DESCRIPCIÓN (Material, reactivo, instrumento, software, hardware, equipo)	CANTIDAD	UNIDAD DE MEDIDA
NA		

7. **MATERIALES, REACTIVOS, INSTRUMENTOS, SOFTWARE, HARDWARE O EQUIPOS DEL ESTUDIANTE:**

DESCRIPCIÓN (Material, reactivo, instrumento, software, hardware, equipo)	CANTIDAD	UNIDAD DE MEDIDA
Computador	1	NA
Python 3.13.5	1	NA
Software IDE (Microsoft Visual Studio Code)	1	NA



Laboratorio 5. Programación con hilos en Python

8. PRECAUCIONES CON LOS MATERIALES, REACTIVOS, INSTRUMENTOS Y EQUIPOS A UTILIZAR:

Utilizar adecuadamente el equipo de cómputo para realizar la práctica de laboratorio.

9. PROCEDIMIENTO, MÉTODO O ACTIVIDADES:

1. Manejo de Hilos en Python (Explicación Detallada)

El manejo de hilos (threads) en Python permite ejecutar tareas de manera concurrente dentro de un mismo proceso. Esto es especialmente útil en aplicaciones que necesitan realizar múltiples operaciones simultáneamente, como acceder a redes, leer archivos, responder a entradas del usuario o controlar dispositivos físicos. En lugar de esperar que cada tarea termine antes de comenzar la siguiente, los hilos permiten que varias tareas se ejecuten "al mismo tiempo", mejorando la eficiencia y la fluidez del programa.

Python incluye la librería estándar threading, que facilita la creación y administración de hilos. Es importante recordar que, debido al GIL (Global Interpreter Lock), Python no ejecuta múltiples hilos simultáneamente en CPUs distintas cuando se trata de operaciones pesadas de CPU. Sin embargo, para operaciones de entrada/salida, threading es ideal. Interactuar con el usuario sin bloquear la aplicación principal.

2. Crear y Ejecutar un Hilo

Ejemplo básico:

```
import threading

def tarea():
    print("Hola desde un hilo")

# Crear el hilo
hilo = threading.Thread(target=tarea)

# Iniciar la ejecución
hilo.start()

# Esperar a que el hilo termine
hilo.join()

print("Finaliza el hilo principal")
```



Laboratorio 5. Programación con hilos en Python

target=tarea: indica la función que ejecutará el hilo.

start(): inicia el hilo.

join(): bloquea hasta que el hilo termina.

3. Crear Hilos con Argumentos

Puedes pasar argumentos a la función del hilo usando args:

```
def saludar(nombre):  
    print(f"Hola {nombre}")  
  
hilo = threading.Thread(target=saludar, args=("Ana",))  
hilo.start()  
hilo.join()
```

4. Hilos como Clases

Otra forma de crear hilos es creando una clase que herede de threading.Thread.

```
class MiHilo(threading.Thread):  
    def run(self):  
        print("Hola desde una clase hilo")  
  
h = MiHilo()  
h.start()  
h.join()
```

5. Ejecutar Varios Hilos

```
import threading  
import time  
  
def tarea(nombre):  
    print(f"Inicia {nombre}")  
    time.sleep(2)  
    print(f"Termina {nombre}")  
  
h1 = threading.Thread(target=tarea, args=("Hilo 1",))  
h2 = threading.Thread(target=tarea, args=("Hilo 2",))  
  
h1.start()  
h2.start()  
  
h1.join()  
h2.join()
```

Ambos hilos se ejecutan al mismo tiempo y finalizan aproximadamente juntos.



Laboratorio 5. Programación con hilos en Python

6. Daemon Threads

Un hilo daemon se ejecuta en segundo plano. Se detiene cuando finaliza el hilo principal.

```
import threading
import time

def tarea():
    while True:
        print("Trabajando en segundo plano...")
        time.sleep(1)

hilo = threading.Thread(target=tarea)
hilo.daemon = True
hilo.start()

time.sleep(3)
print("Fin del programa principal")
```

7. Condiciones de Carrera y Locks

Si varios hilos acceden al mismo recurso (por ejemplo, una variable), puede haber problemas de sincronización:

```
import threading

contador = 0
lock = threading.Lock()

def incrementar():
    global contador
    for _ in range(100000):
        with lock:
            contador += 1

h1 = threading.Thread(target=incrementar)
h2 = threading.Thread(target=incrementar)

h1.start()
h2.start()
h1.join()
h2.join()

print("Contador:", contador)
```



Laboratorio 5. Programación con hilos en Python

with lock: asegura que solo un hilo pueda modificar contador a la vez.

8. Comunicación entre Hilos: Event

El objeto Event permite que un hilo espere una señal de otro:

```
import threading
import time

evento = threading.Event()

def esperar():
    print("Esperando señal...")
    evento.wait()
    print(";Señal recibida!")

t = threading.Thread(target=esperar)
t.start()

time.sleep(2)
evento.set() # Dispara el evento
```

9. Uso de Timer (Temporizadores)

threading.Timer permite ejecutar una función luego de un retardo:

```
from threading import Timer

def saludar():
    print("Hola después de 3 segundos")

temporizador = Timer(3.0, saludar)
temporizador.start()
```

10.Caso Práctico: Cajero Multicliente

```
import threading
import time
import random

def atender(cliente):
    print(f"Atendiendo a {cliente}")
    time.sleep(random.randint(1, 4))
    print(f"{cliente} finalizó su operación")

clientes = ["Ana", "Luis", "Pedro", "Marta"]
```



Laboratorio 5. Programación con hilos en Python

```
hilos = []
for c in clientes:
    t = threading.Thread(target=atender, args=(c,))
    t.start()
    hilos.append(t)

for h in hilos:
    h.join()

print("Todos los clientes han sido atendidos")
```

4. Ejercicio aplicado

Elaborar un programa que permita simular gráficamente una competencia automovilística de diez vehículos. Cada vehículo debe tener **una velocidad aleatoria** y se debe ejecutar mediante un hilo para cada carro (debe haber 10 hilos en el programa). Al final debe hacer una tabla ordenada **con los tiempos** de menor a mayor iniciando por el ganador.

- Se debe utilizar tkinter o PyQt para desarrollar la interfaz gráfica (GUI)
- Cada vehículo debe tener una velocidad aleatoria (puede cambiar constantemente o cuando el vehículo llegue a la salida o a la meta)
- Cada vehículo debe tener un hilo independiente de los demás
- Al final debe hacer una tabla ordenada con los tiempos de menor a mayor iniciando por el ganador.
- Permita que el usuario pueda apostar por algún vehículo antes de que inicie el juego y anuncie si el vehículo seleccionado fue el ganador o no.
- La carrera debe tener un slider que permita acelerar o desacelerar el tiempo general del juego.
- Permita que el usuario programe el número de idas y vueltas (rondas) que deben ejecutar los vehículos.
- La etiqueta de cada vehículo será una imagen, foto, dibujo, etc.



Laboratorio 5. Programación con hilos en Python

11 RESULTADOS ESPERADOS:

- El estudiante debe estar en capacidad de construir y ejecutar aplicaciones en el IDE Visual Studio Code utilizando el lenguaje de programación orientado a objetos Python.
- La aplicación debe programarse utilizando un hilo para cada vehículo
- Se debe implementar con programación orientada a objetos

12 CRITERIO DE EVALUACIÓN A LA PRESENTE PRÁCTICA:

El estudiante debe entregar el programa funcionando, su código fuente y sustentar su funcionamiento.

Criterio	No cumple		Cumple parcialmente	Cumple totalmente
Sustenta satisfactoriamente y responde a las preguntas del docente	0		0.7	1.4
Implementa los algoritmos solicitados	0		1.5	3
Presenta claridad y orden y el código desarrollado es orientado a objetos	0		0.3	0.6
Total				5.0

13 REFERENCIAS

<https://realpython.com/intro-to-python-threading/>

<https://www.codigopiton.com/como-usar-hilos-o-threads-en-python/>